arXiv:2504.13272v1 [cs.SE] 17 Apr 2025

Using LLMs for Library Migration

MOHAYEMINUL ISLAM, University of Alberta, Canada
AJAY KUMAR JHA, North Dakota State University, USA
MAY MAHMOUD, New York University Abu Dhabi, United Arab Emirates
ILDAR AKHMETOV, Northeastern University, Canada
SARAH NADI, New York University Abu Dhabi and University of Alberta, United Arab Emirates

Library migration is the process of replacing a used software library with another library that provides similar functionality. Manual library migration is time-consuming and error prone, as it requires developers to understand the APIs of both libraries, map them, and perform the necessary code transformations. Due to its difficulty, most of the existing automated techniques and tooling stop at the API mapping stage or support a limited set of code transformations. On the other hand, Large Language Models (LLMs) are good at generating and transforming code and finding similar code, which are necessary upstream tasks for library migration. Such capabilities suggest that LLMs may be suitable for library migration. Therefore, in this paper, we investigate the effectiveness of LLMs for migration between Python libraries. We evaluate three LLMs, LLama 3.1, GPT-4o mini, and GPT-4o on PyMigBench, where we migrate 321 real-world library migrations that include 2,989 migration-related code changes. We measure the correctness of the migration results in two ways. We first compare the LLM's migrated code with the developers' migrated code in the benchmark and then run the unit tests available in the client repositories. We find that LLama 3.1, GPT-4o mini, and GPT-4o correctly migrate 89%, 89%, and 94% of the migration-related code changes. respectively. We also find that 36%, 52% and 64% of the LLama 3.1, GPT-4o mini, and GPT-4o migrations pass the same tests that passed in the developer's migration. Overall, our results suggest that LLMs can be effective in migrating code between libraries, but we also identify cases that pose difficulties for the LLM.

Additional Key Words and Phrases: Python, library migration, LLM, code transformation

1 Introduction

Developers often replace one library with another in their applications to improve performance, address security vulnerabilities, or even for licensing compatibility [14, 27]. This process, known as *library migration* [45], is time consuming and error prone [27], as it requires developers to understand the Application Programming Interfaces (APIs) of both libraries, find API replacements, and then perform the needed code transformations [4].

There have been previous efforts towards automating library migration, but fully usable tools for this task are limited. With the exception of a few attempts [33], most of the existing library migration techniques only focus on finding API mappings between the source and target libraries [2, 4, 8, 46, 57], but do not provide support for the actual code transformation.

In recent years, Large Language Models (LLMs) have shown that they can be effective in various software engineering tasks [17, 37, 48], including the necessary upstream tasks for library migration, such as code generation [30, 32, 40], code comprehension [31], and code transformation [51, 53, 58]. This suggests that LLMs may be capable of performing library migration. Accordingly, this paper investigates the effectiveness of LLMs for library migration.

Our investigation focuses on Python, and we use PyMigBench [26], an existing dataset of 321 real-world library migrations containing 2,989 migration-related code changes. The data includes the code before and after the migration, along with detailed descriptions of the migration-related code changes including labeling of the types of changes using an existing taxonomy PyMigTax [25].

Authors' Contact Information: Mohayeminul Islam, University of Alberta, Canada, mohayemin@ualberta.ca; Ajay Kumar Jha, North Dakota State University, USA, ajay.jha.1@ndsu.edu; May Mahmoud, New York University Abu Dhabi, United Arab Emirates, m.mahmoud@nyu.edu; Ildar Akhmetov, Northeastern University, Canada, i.akhmetov@northeastern.edu; Sarah Nadi, New York University Abu Dhabi and University of Alberta, United Arab Emirates, sarah.nadi@nyu.edu.

We use three state of the art LLMs (LLama 3.1 [15], GPT-4o mini [36], and GPT-4o [35]) to perform the migrations. We then evaluate the correctness of the migrations in two ways. First, we compare the LLM-migrated code with the developer-migrated code and report correctness at two granularity levels: the full migration and the individual code changes. We use PyMigTax [25] to identify which types of code changes the LLMs can/cannot handle. Next, for the subset of migrations with available unit tests, we run the available tests to assess the correctness of the migrated code.

We find that GPT-4o performs the best among the three LLMs, while also being the most expensive. It correctly migrates at least one migration-related code change for 94% of the migrations and perfectly migrates 57% of the migrations. At the code change level, GPT-4o correctly migrates 94% of the migration-related code changes. In the test-based evaluation, 64% of the migrations done by GPT-4o pass the same set of tests that passed in the developer's migration. GPT-4o mini closely follows GPT-4o in performance, perfectly migrating 49% of the migrations and 89% of the migration-related code changes. While LLama 3.1 performs the same as GPT-4o mini at the code change level (89% correct), it performs much worse at the migration level, fully migrating only 26% of the migrations compared to 49% by GPT-4o mini.

Our analysis of the code changes using PyMigTax reveals that all three LLMs can perform reasonably well on the majority of the code change types. Specifically, their performance on higher cardinality code change is promising (70%, 79% and 84% correct). However, all three LLMs struggle with code changes that require argument transformation, including changes to the argument value or type. GPT-4o outperforms the other two models in most of the code change types, especially in function call and higher cardinality code changes.

To summarize, our contributions in this paper are as follows:

(1) We investigate the effectiveness of three different LLMs for library migration using an existing dataset of 321 real-world library migrations, PyMigBench, that provides us with labeled migration-related code changes.
(2) We evaluate the correctness of the LLM-based migration in multiple ways: (1) comparing it to the developer changes at both migration and code change levels and (2) using unit tests, which provides a run-time evaluation of the migrated code.
(3) Using PyMigTax [25], we identify which types of code changes the LLMs can/cannot handle.

2 Background and Terminology

2.1 Library Migration

Library migration is the process of updating a software project to replace a used library with another library that provides similar functionality [45]. The *source library* is the library being replaced, and the *target library* is the library that replaces it [45]. When we refer to one instance of a *migration*, this refers to a commit in a repository where a migration happened from a specific source library to a target library [26] (e.g., commit `b0607a26` in repository *openstack/ironic* contains a migration from *retrying* to *tenacity*).

During the migration process, developers replace code segments that use the source library with segments that use the target library. Such changes are called *migration-related code changes*, or just *code changes* for brevity [26]. The lines between the boxes in Figure 3 show code changes. For example, line 15 in Figure 3a is replaced by lines 15-16 in Figure 3b. We adopt the PyMigBench notation x:y to refer to a code change in a given file, e.g., 15:15-16 for the above change.

We use subscripts *pre*, *dev*, and *llm* to denote data before migration, after migration by developers, and after migration by an LLM, respectively. Thus, code_{pre} is the code before migration, code_{dev} after developer migration, and code_{llm} after an LLM migration. Change_{dev} and Change_{llm} denote migration-related code changes by developers and an LLM, respectively.

Table 1. Description of the three models we use in our experiment.

	LLama 3.1	GPT-4o mini	GPT-4o
Used version	3.1-70B-Instruct	2024-07-18	2024-08-06
Model Type	Open-source	Proprietary	Proprietary
# training parameters	70 billion	No official OpenAI info	No official OpenAI info
Context length	128K tokens	128K tokens	128K tokens
Output limit	4096 tokens [1]	16,384 tokens	16,384 tokens
Cost per million input tokens [2]	Free	$0.15	$2.5

2.2 PyMigBench and PyMigTax

PyMigBench is a dataset of real-world Python library migrations [26]. We use the latest version, version 2.2.5, [25] in our experiments. The dataset has 321 migrations and 2,989 migration-related code changes. It includes the detailed description of the code changes, including the line numbers and API names involved in the code changes, which facilitates our evaluation.

PyMigTax is a taxonomy of migration-related code changes [25]. PyMigTax describes a code change based on three dimensions: *(1) Source/target program elements*: the types of program elements involved in the change (e.g.,function call and attribute); *(2) Cardinality*: how many source APIs are replaced with how many target APIs. For example, one-to-many cardinality means one source API is replaced with multiple target APIs; one-to-many, many-to-one, and many-to-many cardinalities are commonly referred to as *higher-cardinality*; and *(3) Properties*: Additional properties to describe the code change (e.g.,element name change when the source and target APIs have different names and argument addition when the target API needs an additional argument to achieve the same behavior). The code change data in PyMigBench is already annotated with the PyMigTax categories. We use these PyMigTax labels to understand the types of code changes LLMs can handle.

3 Experiment Setup

Table 1 shows the three off-the-shelf models we use for our library migration task. The first model is Meta's LLama 3.1 [15], a free and open-source LLM [29]. The other two models are proprietary: OpenAI GPT-4o mini [34] and GPT-4o [34]. GPT-4o mini replaces OpenAI's GPT-3.5 Turbo and is more cost-effective while GPT-4o is the most powerful of the OpenAI models, while also being the most expensive [34]. We use these models to migrate the code in PyMigBench v2.2.5 dataset, which contains 321 real migrations from 300 repositories.

3.1 Data Preparation

We find that 2 of the 300 repositories are no longer public, preventing us from using them and the corresponding 2 migrations in our experiments. We clone the remaining 298 repositories.

For each migration commit, we use PyDriller [44] to extract and store the content of each file that has recorded migration-related code changes, both before and after the migration. During this process, we check whether the files before and after migration are syntactically correct Python code using Python's built-in `AST` module [20]. If the parsing fails, we consider the file to be syntactically incorrect. We find that 5 migrations have at least one file with syntax errors, and we discard these

[1] LLama 3.1 has an input + output limit of 8192 tokens. Our prompt and response are approximately equal, therefore we consider the output limit to be half (4096).

[2] Cost at the time of our experiment: July and August 2024

> The following Python code uses library <source-lib>. Migrate this code to use library <target-lib> instead. In the output, first explain the changes you made. Then, provide the modified code. Do not make any changes to the code that are not related to migrating between these two libraries. Do not refactor. Do not reformat. Do not optimize. Do not change coding style. Provided code:

Fig. 1. LLM Migration Prompt

migrations from our experiments. We conduct our experiments with the remaining 314 migrations containing 2,910 migration-related code changes.

3.2 Migration

For each file having migration-related code changes, we use the LLM prompt template in Figure 1 to migrate the code. The prompt is designed to ensure that the LLM performs focused, controlled migrations while maintaining transparency. By asking the LLM to "explain the changes you made" before providing the modified code, we ensure that the LLM justifies its modifications, which can be useful for a human evaluator reviewing the migration. Additionally, by restricting the LLM from making unrelated modifications, such as refactoring, reformatting, or optimizing the code, the prompt ensures that the migration remains focused on the task of switching libraries, preserving the original coding style and minimizing unnecessary variability across runs.

Since LLMs are known to be non-deterministic [5], we run each migration 10 times for each model. We calculate the difference between the runs to understand the variability in the generated code. We first find the diff between each pair of the 10 runs [13], and determine the total number of lines that are different. To normalize the diff size, we divide the number of different lines by the total number of lines in the file. We find that on average, two runs on the same file are only 6.8%, 4.0% and 4.8% different for LLama 3.1, GPT-4o mini, and GPT-4o, respectively. Therefore, when evaluating the LLMs' results, we randomly select only one run for each model. Note that our artifact contains the LLM's migrations for all 10 runs.

3.3 Migration Evaluation

Our main goal is to assess the ability of LLMs to migrate Python code between analogous libraries. However, validating migration correctness is challenging, because there may be more than one way to correctly migrate a code segment [7]. The LLM's migrations might not exactly match the developer changes in the benchmark, yet still be correct. Considering this, we use two strategies to assess the correctness of the LLM-migrated code through the following research questions.

RQ1 **How similar are the LLM migrations to the benchmark migrations?** In this first evaluation strategy, we consider the changes made by the developer and stored in PyMigBench as the ground truth. Accordingly, we automatically check if the LLM was able to correctly perform all expected changes, while accounting for refactoring and alternative correct changes through manual review.

RQ2 **How many migrations pass unit tests?** The first RQ provides an assessment of the migrations based on automatic and manual *static* analysis, which does not guarantee run-time correctness. To address this, we use a second evaluation strategy, where we use unit tests to evaluate the run-time correctness of the migrated code.

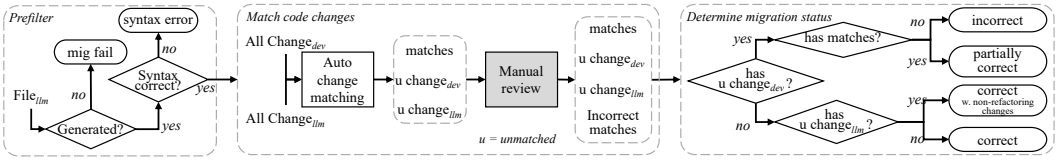
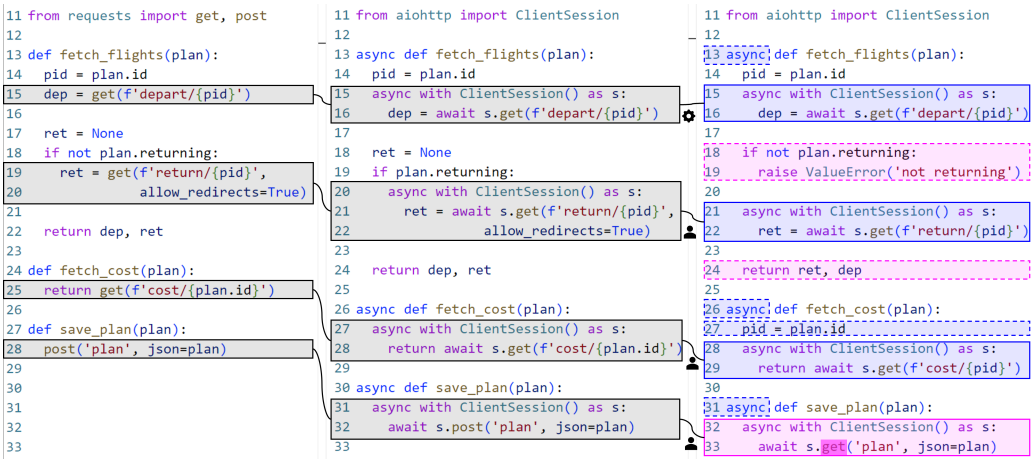


Fig. 2. Benchmark-based evaluation process for a migration. A gray box indicates a manual step.



(a) Code before migration (File$_{pre}$) (b) Developer's migration (File$_{dev}$) (c) LLM Migration (File$_{llm}$)

Fig. 3. A sample migration from *requests* to *aiohttp*

4 RQ1 How similar are the LLM migrations to the benchmark migrations?

4.1 Approach

In this RQ, we assess each LLM's migration correctness by comparing the code changes it made (Change$_{llm}$) with those made by developers (Change$_{dev}$). A migration may involve multiple files, each with multiple migration-related code changes. We evaluate each migration-related code change individually, and then aggregate the results to determine the correctness of the entire migration.

Figure 2 shows how we evaluate the correctness of an LLM migration. We check if the LLM generated a File$_{llm}$ for all File$_{pre}$ in that migration, and if they are free of syntax errors. Next, for each File$_{llm}$, we try to automatically match each Change$_{dev}$ with a corresponding Change$_{llm}$, if available. We manually review any unmatched changes (Uchange$_{dev}$ and Uchange$_{llm}$ in Figure 2) for missed matches. We now detail these steps.

4.1.1 Prefilter. If the LLM fails to produce a File$_{llm}$ from a File$_{pre}$, we label it as a *migration fail* (often this is due to exceeding token limits). If a File$_{llm}$ is generated but has syntax errors, we label it as a *syntax error*. We check syntactic correctness using Python's AST parser. If File$_{llm}$ is syntactically correct, we move to the next step.

4.1.2 Match code changes. To reduce manual comparison of LLM and developer changes, we first automatically match a Change$_{llm}$ with a Change$_{dev}$ if they are syntactically identical. Unmatched developer changes undergo manual review. We use the example in Figure 3 to explain the process.

Table 2. Code change matching results from Figure 3

Change_{dev}	Change_{llm}	Auto results	Manual result
	13:13	Unmatch	Migration related
15:15-16	15:15-16	Exact match	-
	17-18:18-19	Unmatch	Unmatch
19-20:20-22	19-20:21-22	Unmatch	Alternative API
	22:24	Unmatch	Unmatch
	24:26	Unmatch	Migration related
	:27	Unmatch	Refactored
25:27-28	25:28-29	Unmatch	Refactored
	13:13	Unmatch	Migration related
28:31-32	28:32-33	Unmatch	Incorrect API

The original code in File_{pre} (Figure 3a) uses the library *requests* [18] for HTTP requests. The developer migrated this to *aiohttp* [1] (Figure 3b), and the LLM's migration is shown in Figure 3c. Table 2 presents the matching results for this example, which we now explain.

Automatically matching Change_{dev} with Change_{llm}. Our goal in this step is to reduce the number of code changes that require manual review, while making sure that the automated matching is precise, i.e., we do not falsely mark a Change_{llm} to be correct. Therefore, we only consider exact syntactic matches between Change_{dev} and Change_{llm}, while ignoring formatting differences. For a given pair of File_{dev} and File_{llm}, we extract the ASTs, AST_{dev} and AST_{llm}, using Python's AST module [20]. Then for each Change_{dev}, we find the AST nodes from AST_{dev} that represent the Change_{dev} using the API names and line numbers provided by PyMigBench. We then search for nodes in AST_{llm} with the same API names, ensuring they are in the same container (function or class definitions) as the Change_{dev}. Note that the line numbers of a Change_{dev} may be different from the line numbers of a corresponding Change_{llm}, therefore, we cannot only use line numbers to match the nodes. We may find multiple sets of nodes in AST_{llm} that potentially match a Change_{dev}; therefore, to find the correct set of nodes, we first group them based on proximity in the code based on their line numbers. We then convert the two matched node sets into two code strings using the `unparse` function from the `AST` module [21]. If the code strings are exactly the same, we consider the Change_{llm} to be the match for the Change_{dev}.

The Change_{dev} 15:15-16 in Figure 3 uses the `ClientSession()` and the `get()` functions from the *aiohttp* library. In the File_{llm}, we find these functions at lines 15, 16, 21, 22, 28, 29, 32 and 33. The lines 28, 29, 32 and 33 are not in the same container as the Change_{dev} (function `fetch_flights`), so we discard them. We group the nodes based on proximity of line numbers, and find 15:15-16 and 15:21-22 to be potential matches for this Change_{dev}. The code string for the first group is `"async with ClientSession() await s.get(f'return/pid')"` and matches that of the Change_{dev}; hence, we consider this Change_{llm} to be the match for this Change_{dev}. The *Auto results* column in Table 2 shows that only the Change_{dev} 15:15-16 is matched automatically (denoted by a gear symbol in the figure), while the other Change_{dev} and several Change_{llm} remain unmatched.

Manually matching Uchange_{dev}. The LLM might correctly migrate code differently than the developer, such as using alternate variable names or APIs. Thus, we use Meld [52], a three-way diff tool, to manually compare File_{pre}, File_{dev}, and File_{llm} for each file that has at least one Unmatch_{dev}. We use the online API documentation, as well as the source code, of the source and target libraries

to understand the correct API usage, and consider the code changes in the context of the entire file. In the example in Figure 3, we manually find that the Change$_{llm}$ 19-20:21-22 is a correct alternative to Change$_{dev}$ 19-20:20-22, noting that the LLM omitted the default value (`True`) of the `allow_redirects` argument, which the developer included. Change$_{llm}$ 25:28-29 is also a correct replacement of Change$_{dev}$ 25:27-28, where the LLM performed a refactoring that the automatic match did not capture. We also find that the LLM used the function `get` instead of `post` at line 33, making Change$_{llm}$ 28:32-33 an incorrect replacement of Change$_{dev}$ 28:31-32.

Manually finding non-refactoring Uchange$_{llm}$. After trying to match all changes in Change$_{dev}$, we may find that some Change$_{llm}$ remain unmatched. Our aim is to categorize these as refactoring (not altering program semantics) or non-refactoring (potentially affecting code behavior). If a Change$_{llm}$ is a refactoring, we remove it from Uchange$_{llm}$, as it is does not impact migration correctness.

PyMigBench records API usage line numbers, offering precise insights into the migration-related code changes made by the developer. While it captures specific instances effectively, it is worth noting that it does not capture additional changes indirectly related to the migration, i.e., lines that do not have a target library API usage. For example, adding `async` to function definitions (lines 13, 26, and 31) due to asynchronous calls introduced by migration is not captured. We remove these changes from the set of unmatched Change$_{llm}$, since they correctly relate to the migration.

In the example, the LLM introduced handling of a `ValueError` (line 19). Since this impacts program behavior, we mark it as a non-refactoring change. Similarly, we label the swapped variables on line 24 as non-refactoring. Overall, Change$_{llm}$ 17-18:18-19 and 22:24 stay unmatched, while 13:13, 24:26, :27, and 27:31 get removed from the unmatched set.

To ensure the quality of the manual review, we have two annotators independently review the changes in a given file. We first do so for 194 code changes from one model, and measure the agreement using Cohen's Kappa [12], but we do not reach substantial agreement. Therefore, we discuss and resolve disagreements and update our coding guideline. We then review another 114 code changes, and achieve a Cohen's Kappa score of 0.83 and 0.73 for two pairs of reviewers, indicating almost perfect and substantial agreement, respectively [28]. We therefore review the remaining code changes with only one annotator per code change.

Determine code change status. At this point, we have finished the matching process and can determine whether the LLM correctly migrated each Change$_{dev}$ in the benchmark. Based on its matching to a Change$_{llm}$, we assign each Change$_{dev}$ with one of the following categories.

- *Correct*: The LLM correctly migrated Change$_{dev}$.
  - *Exact match*: The Change$_{llm}$ is syntactically the same as the corresponding Change$_{dev}$. e.g., Change$_{dev}$ 15:15-16 and Change$_{llm}$ 15:15-16 in Figure 3.
  - *Alternative API*: Change$_{llm}$ uses different APIs (e.g., a different function) or arguments, but it is a correct alternate replacement of Change$_{dev}$. e.g., Change$_{dev}$ 19-20:20-22 and Change$_{llm}$ 19-20:21-22 in Figure 3.
  - *Refactored*: The LLM uses the same APIs as the developer, but either the developer or the LLM made some refactoring changes. For example using a different variable name or a different coding style. e.g., Change$_{dev}$ 25:27-28 and Change$_{llm}$ 25:28-29 in Figure 3.
- *Incorrect*: The LLM incorrectly implemented the migration change, or did not migrate it at all.
  - *Incorrect API*: The LLM used incorrect APIs (e.g., wrong function), or correct APIs but incorrect arguments. e.g., the LLM used `get()` instead of `post()` in Change$_{llm}$ 28:32-33 in Figure 3.
  - *No attempt*: The LLM left the source APIs as is, without attempting to migrate them.
  - *Removed incorrectly*: The LLM removed a part of the code that should not have been removed. This includes the code that should either remain as it is, or should be migrated.

Table 3. Correctness of migrations by the LLMs.

	LLama 3.1		GPT-4o mini		GPT-4o	
Status	# Migs	% Migs	# Migs	% Migs	# Migs	% Migs
At least partially correct						
Correct	83	26%	154	49%	179	57%
Correct with non-refactoring changes	28	8.9%	53	17%	55	18%
Partially correct	48	15%	84	27%	62	20%
Subtotal	*159*	*51%*	*291*	*93%*	*296*	*94%*
Fully incorrect						
Incorrect	7	2.2%	6	1.9%	2	0.6%
Syntax error	16	5.1%	6	1.9%	3	1.0%
Response failure	132	42%	11	3.5%	13	4.1%
Subtotal	*155*	*49%*	*23*	*7.3%*	*18*	*5.7%*
Total migrations	314		314		314	

4.1.3 Determine migration status. Based on the individual matched code changes, we determine the overall correctness of a migration as follows:

- *Response failure*: The LLM could not generate a File_{llm} for at least one File_{pre}, because of limitations imposed by the model, such as token limit or API timeout.
- *Syntax error*: The LLM generated a File_{llm} for all File_{pre}, but at least one File_{llm} has syntax errors. The below statuses are assigned only if there are no syntax errors in any of the File_{llm}.
- *Incorrect*: The LLM could not correctly migrate any of the changes marked in PyMigBench. We assign this status when all Change_{dev} across the migration remain unmatched.
- *Partially correct*: The LLM correctly migrated only some of the changes marked in PyMigBench. We assign this status when there are only some unmatched Change_{dev}.
- *Correct with non-refactoring changes*: The LLM correctly migrated all the changes marked in PyMigBench, but has also done some non-refactoring changes. We assign this status when there are no unmatched Change_{dev} in any file, but there are some remaining unmatched Change_{llm}.
- *Correct*: The LLM correctly migrated all the changes from PyMigBench for this migration, without any non-refactoring changes. We assign this status when there are no unmatched Change_{dev} or Change_{llm} left in any of the files after the matching process.

4.2 Findings: Migration level correctness

Table 3 shows the results for the migrations we evaluate. We use the names LLama, Mini, and 4o to refer to LLama 3.1, GPT-4o mini, and GPT-4o, respectively, for brevity. We find that 4o performs the best with 94% of the migrations having at least one correct code change, closely followed by Mini with 93%. However, when considering fully correct migrations, GPT-4o significantly outperforms Mini, with 57% compared to 49%. LLama has the lowest percentage of correct migrations, with only 26% being fully correct and 51% having at least one correct code change.

The table shows that LLama has a high proportion of *response failure*, especially compared to the other models. This status can be assigned for two reasons: either the prompt or the completion exceeded the token limit, or the model could not complete the migration within the time limit. The second reason is very rare: we observe it only once for the GPT-4o mini model for a specific file, and even after re-running it twice, the same timeout occurred. The first reason is the primary cause

Table 4. Correctness of migration-related code changes.

	LLama 3.1		GPT-4o mini		GPT-4o	
Status	# CCs	% CCs	# CCs	% CCs	# CCs	% CCs
Correct						
Exact match	481	53%	1,226	62%	1,274	64%
Alternative API	295	32%	466	24%	553	28%
Refactored	32	3.5%	52	2.6%	40	2.0%
Subtotal	*808*	*89%*	*1,744*	*89%*	*1,867*	*94%*
Incorrect						
Incorrect API	93	10%	169	8.6%	107	5.4%
No attempt	1	0.1%	38	1.9%	12	0.6%
Removed incorrectly	9	1.0%	15	0.8%	10	0.5%
Subtotal	*103*	*11%*	*222*	*11%*	*129*	*6.5%*
Evaluated code changes	911	100%	1,966	100%	1,996	100%
Excluded code changes	1,999		944		914	
Total code changes	2,910		2,910		2,910	

of LLama's results, since it has a smaller token limit compared to the other two models (see Table 1), making it more likely to exceed the limit. LLama also produces more syntax errors compared to the other two models (5.1% vs. 1.9% and 1.0%).

Overall, we find that the LLMs we experimented with can correctly migrate a high portion of the migrations, with 4o performing the best among the three models, while Mini also performing almost similarly well. LLama, however, performs relatively worse than the other two models, specially due to lower output token limit.

4.3 Findings: code change level correctness

We now present the correctness results at the individual code change level. The 314 migrations we use for our evaluation contain a total of 2,910 code changes. However, when we report the correctness of each code change, we have to first exclude code changes in the migrations that have syntax errors in their $code_{llm}$ or the migrations with response failures. This excludes 1,999 code changes from LLama, 944 from Mini, and 914 from 4o. This leaves us with 911, 1,966, and 1,996 code changes to evaluate for the three LLMs, respectively.

We first discuss the overall correctness of the code changes, then discuss the correctness across different PyMigTax categories.

4.3.1 Overall code change correctness. Table 4 shows the overall correctness of the LLMs's migrations at the code changes level. 4o performs the best with 94% of the code changes being correct. Interestingly, Mini and LLama both perform equally well at the code change level, with both having 89% correct code changes. This is in contrast to the migration level correctness, where Mini performs better than LLama (49% vs. 26%). This discrepancy can be explained by the fact that Mini has attempted more code changes compared to LLama (1,966 vs 911) as the latter has more syntax errors and response failures. To confirm whether this inflates the performance of LLama, we compare the correctness of 892 code changes from 162 migrations that are common between all three models. We find that LLama, Mini, and 4o have 89%, 88%, and 94% correct code changes

Table 5. Correctness of code changes across PyMigTax dimensions.

	Frequency in PyMigBench			LLama 3.1		GPT-4o mini		GPT-4o	
PyMigTax labels	% Migs	% LPs	% CCs	# Eval	Correct	# Eval	Correct	# Eval	Correct
Program elements									
Function call	71%	78%	60%	518	88%	1,093	87%	1,092	93%
Import	99%	99%	24%	276	95%	580	94%	593	98%
Decorator	12%	10%	14%	197	87%	335	90%	353	87%
Attribute	17%	25%	6.1%	63	83%	103	78%	103	89%
Type	5.3%	10%	1.8%	12	75%	34	85%	34	88%
Exception	5.0%	10%	1.0%	17	65%	29	86%	29	93%
Function reference	1.2%	3.0%	0.4%	3	33%	12	58%	12	100%
Properties									
No properties	18%	27%	8.8%	43	84%	150	88%	150	97%
Element name change	59%	66%	34%	398	84%	773	85%	771	90%
Argument addition	29%	37%	19%	150	85%	244	80%	244	88%
Argument deletion	21%	28%	12%	175	85%	234	87%	234	89%
Argument transformation	25%	28%	11%	115	77%	201	76%	220	77%
Parameter addition to decorated function	4.0%	0.7%	4.2%	110	87%	127	95%	127	90%
Argument name change	5.3%	8.2%	3.7%	20	60%	58	59%	58	84%
Async transformation	4.0%	3.0%	1.7%	35	83%	50	90%	50	92%
Output transformation	5.0%	11%	1.6%	25	88%	49	78%	49	92%
Cardinality									
One-to-One	62%	73%	52%	408	86%	877	84%	894	91%
One-to-Zero	16%	19%	10%	99	99%	290	99%	290	100%
Zero-to-One	11%	13%	6.6%	34	97%	55	87%	55	89%
Higher Cardinality	*28%*	*34%*	*7.0%*	*94*	*70%*	*164*	*79%*	*164*	*84%*
One-to-Many	16%	17%	4.8%	61	69%	103	78%	103	80%
Many-to-One	9.3%	18%	1.4%	16	62%	34	82%	34	91%
Many-to-Many	6.5%	8.2%	0.9%	17	62%	27	82%	27	91%
Total	321	134	2,989	911	89%	1,966	89%	1,996	94%

respectively, which very closely match their respective overall correctness rates. This suggests that LLama's evaluation using fewer code changes does not inflate its performance.

4.3.2 Code change correctness by category. To understand whether some types of code changes are more difficult to migrate than others, Table 5 shows the distribution of the correctness of the code changes across the different PyMigTax dimensions and labels. The first set of columns, Frequency in PyMigBench, show how often each PyMigTax dimension appears in PyMigBench, in terms of proportion of migrations (Migs), library pairs (LPs), and code changes (CCs). For example, 71% of the 321 migrations in PyMigBench involve function calls. This gives us information on how common each category is in practice, and also provides a perspective on the impact of each category on the overall performance of the LLMs. This information is the same for each LLM since this describes the data in PyMigBench. The next three column groups show the performance of each of the three models. Since each LLM has a different number of evaluated code changes, due to the

varying performance in terms of failed migrations due to exceeded token limit or due to syntax errors, we first show the number of evaluated code changes from each category (#Eval) and then show the proportion of these code changes that are correct (Correct). For example, the *Function call* row under the *Program elements* group shows that our experiments with LLama include 518 code changes that involve a function call as either a source or a target program element, and that 456 of these 518 code changes (88%) were correctly migrated. Note that the cells in a column in the row groups may not add up to the total at the bottom, as a single code change can span multiple categories, also some code changes may not have any properties or cardinality.

Program elements. Function calls are the most common program elements in the migrations included in PyMigBench, representing 60% of the all code changes. This category is also the second most common in terms of number of migrations and library pairs that include it (71% and 78% respectively). All three LLMs correctly migrate a high proportion of these changes, with 88%, 87%, and 93% of the function calls being correctly migrated by LLama, Mini, and 4o respectively.

Almost all migrations and library pairs need to migrate import statements, with the exception of migrations from PIL [10] to Pillow [11], where the latter is a fork of the former, and keeps the same import name. All three LLMs perform well in migrating import statements, specially 4o getting almost all of them correct (98%).

Mini slightly outperforms the other models in migrating decorators (90% vs. 87% and 87%), however, does worst in migrating attributes (78% vs. 83% and 89%). 4o performs better than other models in migrating types, exceptions and function references, followed by Mini and then LLama. LLama and Mini both struggle with migrating function references (33% and 58% correct), while 4o correctly performs all such code changes. While this last category shows most discrepancy between the models, it is important to note that this program element is present in only 0.4% of the code changes, therefore does not affect the overall performance significantly.

Properties. The property *no properties* indicates that the source and target APIs are identical, and hence no changes are required. While 4o correctly leaves almost all of these APIs unchanged (97%), LLama and Mini incorrectly changed a significant portion of them (84% and 88% correct respectively).

Element name change is the most common property in terms of the proportion of migrations, library pairs and code changes that involve this property. This indicates that the target APIs commonly bear different names from the source APIs. All three LLMs perform well in migrating element name changes, with 84%, 85%, and 90% of these changes being correctly migrated by LLama, Mini, and 4o respectively.

All three LLMs performed better or equally well in code changes having argument deletion compared to those with argument addition, which is expected as adding an argument requires the LLM to find a suitable replacement, while deleting an argument is a simpler task. The Argument transformation property indicates that an argument requires various changes, such as changing the type or value. All three LLMs correctly migrate about three-quarters of the argument transformation changes. We discuss specific failure instances in section 6.

The property parameter addition to decorated function is found in migrations from or to the click library [38], where the decorators require adding a parameter to the decorated function. All three LLMs perform well in migrating these changes; interestingly, Mini performs even better than 4o (95% vs. 90%). Note that Mini also performed better than 4o in migrating decorators, which are also common in the click library. These are the only two taxonomy categories where 4o performs worse than any other model.

Argument name change applies when an argument representing the same semantics has different names in the source and target APIs. While 4o performs reasonably well in migrating these changes (84%), LLama and Mini fail frequently with them (60% and 59% correct).

Overall, we find that all three LLMs perform well in migrating code changes with most of the properties, except for argument transformation where all three models struggle, and argument name change where LLama and Mini struggle.

Cardinality. One-to-One code changes are the most common, representing 52% of all code changes in PyMigBench, and all three LLMs perform well in migrating them. One-to-Zero code changes are the ones where a source API needs to be removed, but no target needs to be added. Because it is a simple delete operation, all three LLMs migrate all or almost all of these changes correctly. Its counterpart, Zero-to-One code changes, are the ones where a target needs to be added, but no source needs to be removed. Interestingly, LLama performs the best in migrating these changes (97% correct), making this the only category where LLama performs better than the Mini and 4o (87% and 89% correct respectively).

Previous research has shown that higher-cardinality code changes are generally considered difficult compared to one-to-one code changes [2, 24, 47, 57]. The PyMigBench dataset, however, shows that 34% of the library pairs required migrating at least one higher-cardinality code change. We find that the LLMs are reasonably successful at migrating them, with 70%, 79%, and 84% correctness rates for LLama, Mini, and 4o respectively.

5 RQ2 How many migrations pass unit tests?

5.1 Approach

In this second evaluation, we assess a migration's correctness using the unit tests available in its repository. A test-based evaluation ensures run-time correctness and that the migration achieves the expected behavior. If the tests that pass on code_{dev} also pass on the code_{llm}, we consider the migration successful. For this, we first prepare the code for testing, then set up a virtual environment, and then run the tests. Finally, we compare the test results of developer's migration and LLM's migration to determine the correctness of the migration.

5.1.1 Preparing the code. We need two versions of the code where we run the tests: the version that the developer migrated (code_{dev}) and the version that the LLM migrated (code_{llm}). We first make a copy of the code after the developers' migration (code_{dev}) from the repository. We then make another copy of code_{dev}, but replace all File_{dev} with File_{llm}; we refer to this as code_{llm}. The idea is that code_{llm} is basically the version of code_{dev} where the LLM has done the migration on behalf of the developer.

5.1.2 Setting up the virtual environment. To set up a virtual environment, we need to identify the Python version used in the client repository. If this information is available in the `setup.py` file, we use that version. Otherwise, we resolve the version based on the migration commit date as follows. We first identify the release dates of all minor versions of Python (3.6, 3.7 etc). Then we find the latest release date that is before the migration commit date. This is the latest Python version that was available at the time of the migration; we use this version to create the virtual environment. For example, if the resolved Python version is 3.7, we run the command `python3.7 -m venv`.

Next, we install the code dependencies, both those specified in project configuration files or in the requirements file. We first run the command `python -m pip install <code-path>`, which installs the dependencies specified in `setup.py` and `pyproject.toml` files. We then also install any dependencies specified in a requirements files. However, the exact version of the dependencies may not be specified in those files. In such cases, we look up the version history of the dependency

Table 6. Correctness of migrations by the LLMs based on unit tests.

	LLama 3.1		GPT-4o mini		GPT-4o	
Status	# Migs	% Migs	# Migs	% Migs	# Migs	% Migs
Correct	9	36%	13	52%	16	64%
Partially correct	1	4.0%	5	20%	2	8.0%
Incorrect	15	60%	7	28%	7	28%
Evaluated	25		25		25	

on PyPI and install the latest version available at the migration commit date, using the same way we use to resolve the Python version.

5.1.3 Running the tests and coverage. Once the virtual environment is ready, we run the unit tests available using the command `python -m pytest` on code_{dev}. This command runs all the tests in the repository. If the run has errors, we read the error log and try to identify the cause. For the projects that can potentially be fixed, we maintain a configuration file where we record project specific configurations or commands we need. Specifically, we record (1) the required Python version, (2) the location of the requirements files in the repository (3) versions of specific libraries (4) location of the test files (5) special test commands (6) whether to run the tests as a single suite from the root or each test folders separately. The file where we record these configurations is included in the artifact. After finishing the configuration, we set up the virtual environment again with the updated configuration.

After finalizing the configuration, we run the tests on code_{dev} while measuring coverage. Out of the 314 migrations, we find that 218 migrations have at least one test. Among these, 172 migrations raise errors when running the tests, therefore, we could not use them for the evaluation. The failures are primarily due to missing dependencies, which we were unable to resolve, specially for older projects. The remaining 46 migrations have tests that we are able to successfully run on code_{dev}.

For the tests to be useful to validate migration, they must cover the migration-related code changes. We use the coverage report to identify if the tests cover the migration-related code changes. We find that only 27 migrations have at least one test that covers the code changes, and among them, 25 migrations have at least one test that pass in code_{dev}. We run the entire test suite on code_{llm} for these 25 migrations for each model using the same randomly selected run that we used for evaluation in RQ1.

5.1.4 Determining migration status. For each model, we now compare the test results between code_{dev} and code_{llm} and assign the following statuses to the migrations:

- *Correct*: All passing tests in code_{dev} also pass in code_{llm}.
- *Partially correct*: Some of the tests that pass in code_{dev} pass in code_{llm}, but the others fail or raise runtime errors.
- *Incorrect*: None of the passing tests in code_{dev} pass in code_{llm}, they all fail or raise runtime errors.

5.2 Findings

Table 6 shows the result of the evaluation using unit tests for the 25 migrations that have tests covering the migration-related code changes. 4o has the highest percentage of correct migrations, with 64%, followed by Mini with 52%, and LLama with 36%. Mini has more partially correct

```python
def parse_commands():
  p = ArgumentParser()
  p.add_argument('src', type=Path)
  p.add_argument('dst', type=Path)
  p.add_argument("usedir",
                 action='store_true')
  return p.parse_args()

def main():
  arg = parse_commands()
  convert(arg.src, arg.dst,  arg.use_dir)
```

```python
@argument("src",
          type=Path())
@argument("dst", type=Path())
@argument("usedir", type=bool)
def main(src, dst, usedir):
  convert(src, dst, usedir)
```

Fig. 4. 4o correctly migrating from *argparse* to *click*, libraries using different API styles.

```python
tw_api = Twitter(
    auth = OAuth(oauth_token, oauth_secret,
          consumer_key, consumer_secret))
```

```python
auth = OAuthHandler(consumer_key, consumer_secret)
auth.set_access_token(oauth_token, oauth_secret)
tw_api = tweepy.API(auth)
```

twitter to tweepy, cloudbotirc/cloudbot/f8243223/twitter.py

Fig. 5. LLama correctly migrating a many-to-many code change

migrations than 4o and LLama, leading it to have an equal number of incorrect migrations as 4o (28%). LLama, on the other hand, has 60% incorrect migrations, which is significantly higher than the other two models.

6 Discussion

6.1 Are LLMs Suitable For Library Migration?

6.1.1 The Good. We start with the promising side of using LLMs for library migration. In addition to the high overall correctness of individual code changes and migrations in RQ1, our evaluation reveals that the evaluated LLMs can migrate code changes that were traditionally known as difficult to migrate [54]. We discuss some examples.

Different API styles. The library *argparse* [19] provides API functions for parsing command line arguments, while *click* [38] provides decorators. Figure 4 shows a migration example between these libraries. Despite the completely different API styles, the three LLMs correctly migrate 94% of code changes between this library pair.

Higher cardinality migrations. As shown in RQ2, LLama, Mini, and 4o successfully migrated 70%, 79%, and 84% of the higher cardinality code changes, respectively, which the literature always viewed as complex migrations [3, 54]. Figure 5 shows an example of a many-to-many migration from *twitter* to *tweepy* done by LLama. In addition to correctly replacing the functions `Twitter()`, and `OAuth()` with `OAuthHandler()`, `set_access_token()`, and `API()` with correct arguments, LLama also splits the code into multiple statements, making the code more readable.

Inferring changes outside of the source and target APIs. The migration in Figure 6 adds the `async_mode` argument to the `SocketIO()` function. This is interesting because the API is from the *Flask-SocketIO* library, which is not the source (*eventlet*) or target (*gevent*) libraries. However, the `SocketIO` function works with several libraries, *eventlet* being the default[3]. This means, if 4o did not explicitly set the `async_mode` argument to `gevent`, the code would break after the migration.

6.1.2 The Bad. We also analyze cases where the LLMs did not correctly migrate the code and present notable observations.

[3] https://flask-socketio.readthedocs.io/en/latest/api.html#flask_socketio.SocketIO

```
4 socketio = SocketIO(app)
5 bootstrap = Bootstrap(app)
```

```
4 socketio = SocketIO(app, async_mode='gevent')
5 bootstrap = Bootstrap(app)
```

Fig. 6. 4o performing a correct inferred change

```
109 @retry(stop_max_attempt_number=5,
    wait_fixed=2000)
110 def delete_bucket(name):
```

```
108 @retry(stop=stop_after_attempt(5),
    wait=wait_fixed(2))
109 def delete_bucket(name):
```

```
109 @retry(stop=stop_after_attempt(5),
    wait=wait_fixed(2000))
110 def delete_bucket(name):
```

Fig. 7. Mini filing in argument transformation.

Failing to handle argument transformation. Figure 7 shows an example where the source library *retrying* [23] expects the time arguments in milliseconds, while the target library *tenacity* [42] expects them in seconds. The dataset includes 28 instances of this transformation, and all three LLMs failed to handle it in most cases, except for 4o, which handled three instances correctly, while the other two models correctly handled one instance each. This suggests that while the LLMs can map parameters, differences in expected format or units may not be properly managed.

Replacing exception type. We find cases where the LLM replaces generic exception types with specific ones, which can potentially fail to catch the same exceptions as the original code. This occurs in both migration-related and non-migration-related changes. For example, the LLM replaces an original `Exception` with `ValueError`. While using more specific exceptions is considered good practice [6], it does change the behavior of the code. Since modern development tools are often equipped to suggest better exception handling, it may be safer for an LLM-based migration tool to avoid changing existing exception types during a migration.

6.1.3 The Ugly. We also observe the following problematic LLM behavior.

Extra Changes. LLMs are known for not always following the instructions provided in the prompt [55]. Despite our prompt explicitly asking the LLMs to avoid any refactoring or non-migration changes, we find cases where an LLM, particularly LLama, completely removes parts of the code that do not include any source APIs. We speculate that this behavior may stem from the LLM attempting to optimize output size due to token limitations. Silently deleting code can be dangerous for the overall functionality of the system, and led to test failures in RQ2 for many cases.

Similarly, the LLMs sometimes refactor code that is not related to the migration, adopting a different style. While these improvements can be beneficial, they are unrelated to the migration task at hand. Future work could explore the use of an interactive IDE plugin that confirms the code changes before applying them, potentially allowing the developer to directly edit the changes.

While less "dangerous", we also find that the LLMs remove code comments and docstrings from the source code, even though the comments remain relevant after migration.

Large file handling. We observe that large files pose a challenge to LLama, given its smaller context size. Additionally, we observe that even the other two models often migrate an initial part of a large file but then incorrectly state that the remaining part does not require any changes.

6.2 Differences in Evaluation Setup

While the scope of RQ2 is limited due to the availability of the tests, we do find cases where the migrations are correct in RQ1 but incorrect in RQ2, or vice versa. To better understand these discrepancies, we manually inspect migrations with different results and provide some examples.

```
4  import ipaddr
5
6  def IP(ipaddress):
7    a = ipaddr.IPNetwork(ipaddress)
```

```
4  import ipaddress
5
6  def IP(ip):
7      a = ipaddress.ip_network(ip)
```

```
4  import ipaddress
5
6  def IP(ipaddress):
7    a = ipaddress.ip_network(ipaddress)
```

ipaddr to ipaddress, google/capirca/eb768ea7/capirca/lib/nacaddr.py

Fig. 8. Example of function parameter shadowing import name.

6.2.1 Library version compatibility. When evaluating RQ1, we mark a code change to be correct if it is correct as of the latest version of the library. However, in RQ2, we install the version the developers specified, and if it is not available, we resolve the library based on the migration commit date (see subsection 5.1). Since we do not ask for a specific version number in the prompt, we notice that the LLMs usually pick APIs from the latest version of the target library. In some cases, this version discrepancy led to runtime errors when running tests in LLM's code, which we report as incorrect in RQ2.

6.2.2 Function parameter shadowing import name. Notice in Figure 8 that the function parameter in the function `IP` on line 6 (File$_{pre}$) is named `ipaddress`, the same as the target library. The developer changed the parameter name to `ip` in File$_{dev}$. However, while Mini correctly replaces function `IPNetwork` with `ip_network`, it retained the original parameter name, causing it to shadow the new import name and leading to test failures. 4o changed the parameter name to `ipaddress_str` to avoid the shadowing issue.

6.2.3 Developer doing non-refactoring changes. Recall that in RQ1, we find cases where the developer makes non-refactoring changes in the same commit as the migration while the LLM only does the migration it is expected to do. We count this as a correct migration in RQ1. However, given the changed behavior (which is often reflected in the tests), the tests fail in RQ2.

Overall, the discrepancies show that both evaluation techniques provide different perspectives on the migration correctness.

6.3 Understanding the cost of LLM-based migration

The 314 migrations we use in our experiments involve a total of 258,437 lines of code, or 258.0 KLOC. The 10 runs per model makes it 2584.0 KLOC per model. The total cost of running our experiments was approximately $15 for GPT-4o mini and $420 for GPT-4o, which equates to $0.01 and $0.16 per KLOC, respectively. OpenAI has since reduced prices, so current costs would be lower. LLama 3.1 is free and open-source, though we used a pro subscription of *Hugging Face* (US$9/month[4]) to avoid any installation issues. While library migration is an infrequent task in a project's lifetime, it requires significant developer effort when it happens. The cost of a developer's time can easily exceed the cost of using an LLM, even GPT-4o, which can justify the cost of LLM-based tooling.

7 Related work

7.1 Traditional library migration techniques

The majority of automated techniques for library migration focus only on identifying analogous APIs between source and target libraries. Some techniques identify analogous APIs by analyzing *diffs* in migration commits using static analysis [2, 4, 46] and machine learning [3] approaches. Other efforts overcome the need for real migrations dataset by using techniques like unsupervised learning [8] and natural language processing (NLP) of documentation [33].

[4]https://huggingface.co/pricing

To the best of our knowledge, SOAR [33] is the only library migration technique that applies migration changes to the client code, using program synthesis. SOAR uses unit tests to guide the synthesis process and evaluate the correctness of the migrated code. SOAR is evaluated on two pairs of deep learning libraries, focusing on the migration of neural network models. The nature of these APIs allows SOAR to check the state of the program after each API call, which is used to guide the synthesis process. Thus the authors are able to create their own test cases that check the state of the neural network after each API call. Additionally, given the limited data set, their technique relies on the specific format of the error messages of these libraries to further guide the synthesis process, requiring extensive changes and technique adaptations for running SOAR on any new libraries (or even library domains). In contrast, due to the diversity of the libraries we evaluate on, we use the unit tests existing in the repositories to evaluate the correctness of the migrated code. SOAR supports one-to-many code changes, but only for a small set of pre-determined APIs. The authors report that SOAR times out for 20% of the evaluated migrations, even for small code snippets (the largest being 183 lines of code). Our evaluation suggests that LLMs may be able to handle higher-order transformations out of the box while also handling larger code snippets.

7.2 LLM-based library migration

Zhou et al. [59] proposed HaPiM, a combination of static analysis based API mapping and LLM-based transformation technique. They first train a machine learning model capable of API mapping, and then use the API mappings to guide an LLM in code transformation. Their approach outperforms MigrationMapper [4] and GPT-3.5 Turbo on the BLEU [39] and CodeBLEU [41] metrics.

Their study has similarities to our work but has several limitations. First, they evaluate their approach on only five Java library pairs, while our evaluation covers 134 Python library pairs from 34 application domains. Second, to build API mapping, they require a set of existing migrations in open-source projects, which previous studies identify as a limitation [9, 33]. While an LLM essentially learns from existing data, our experiments do not require an additional API mapping learning process. Third, they compare their performance with other Java migration techniques using the BLEU [39] and CodeBLEU [41] metrics, which are shown to not match human evaluations of code generation tasks [16], such as library migration. They also do not compile or run the produced migrations. In contrast, we evaluate the correctness of the migrated code using PyMigBench [26] and PyMigTax [25], additionally leveraging unit tests to evaluate runtime behavior where available. Finally, they do not analyze the types of code changes that their approach can handle.

7.3 Code transformation using LLM

Language migration, i.e., converting code from one programming language to another, is a code transformation task similar to library migration. Wang et al. developed CodeT5 [50], and later CodeT5+ [49], and showed that these LLMs outperform other existing baselines in language migration tasks. Zhang et al. [56] developed Codeditor, another LLM, specifically for language migration which outperformed all models they compare with, including ChatGPT, CodeT5, CodeT5+. These results show that LLMs fine-tuned for specific code transformation tasks can perform better than general-purpose LLMs. Therefore, our already promising results may even be a lower bound on how effective LLMs may be for the library migration task.

In principle, library migration can be seen as a refactoring task, which previous work also used LLMs for. Shirafuji et al. [43] demonstrate the use of GPT-3.5 to perform refactoring [22] tasks. They use unit tests to evaluate the refactoring, similar to our approach in RQ2. They find that the unit tests pass in 96% cases, and the refactored code has on average 17% less cyclomatic complexity and is 26% shorter than the original code.

Overall, our work is unique w.r.t the existing LLM evaluations for code transformation tasks in the following ways: (1) We evaluate at code change level granularity, which is more fine-grained than the file-level evaluation used in the previous studies, and allows more meaningful comparisons than BLEU-based metrics [16]. (2) At code change level, we also analyze which type of code changes the LLM can better handle, using a state-of-the-art PyMigTax [25]. (3) Additionally, we evaluate the correctness of the migrated code using unit tests where available, which gives a more concrete evaluation of the runtime behavior of the migrated code.

8 Threats to Validity

8.1 Internal validity

Our RQ1 results rely on the recorded data in PyMigBench, which may contain errors. The PyMigBench authors, however, manually vetted each code change to ensure correctness. During our manual evaluation, we found a few cases where the data was incorrectly labeled. Such cases were eventually corrected in PyMigBench.

8.2 Construct validity

In RQ1, we analyze the code change categories based on the PyMigTax category of the Change_{dev}, not of the Change_{llm}. The categories can differ if the LLM produces a different code change than the developer. Categorizing the LLM changes would require additional manual effort following the previous work by Islam et al. [25], which is beyond the scope of this work. That said, our observation is that the code/API differences in most cases are minor, and would not yield in a different PyMigTax category.

LLMs have a risk of memorization, especially since we evaluate on publicly available repositories. However, none of the File_{llm} were identical copies of File_{dev}, which suggests that the LLM is not reciting the code of these files.

The libraries included in the PyMigBench dataset are all sufficiently old libraries, which means that the LLMs have likely seen them before, allowing the LLMs to perform migration out-of-the box. For the newer libraries that the LLM does not know about, additional information, such as documentation, may be required to guide the LLM in the migration process. This can be an interesting avenue for future work.

8.3 External validity

We use three off the shelf LLMs, and using an LLM specifically tuned for the library migration task could potentially yield better results. However, the goal of this work is not to develop a migration tool, but to understand the challenges and opportunities of using LLMs for library migration. Our results may be viewed as a lower bound of the performance of potentially more specialized models.

Our results may not generalize beyond Python. We choose Python, because it is a popular language, and because we did not find a dataset that has the same detailed characterization of migration-related code changes for other languages.

The PyMigBench dataset may not be representative of all libraries available in the Python ecosystem. However, the dataset contains a diverse set of 202 libraries from 34 different application domains, which we believe is a good representation of the Python ecosystem.

9 Conclusion

This paper presented an empirical study of using LLMs for library migration in Python. Specifically, we use LLama 3.1, GPT-4o mini, and GPT-4o to migrate 314 migration commits across 294 client repositories. We compared the LLM changes to the developer changes recorded in PyMigBench

for these migrations. We find that GPT-4o performs the best, and correctly migrates 94% of the individual code changes. At the migration level, it perfectly migrates 57% of the migrations, and 94% of them have at least one correctly migrated code change. We also run unit tests for a subset of the migrations, and find that 64% of the migrations by GPT-4o have the same sets of tests passing in the developer's migration and the LLM's migration. A much lower cost LLM GPT-4o mini and the free LLama 3.1 also perform well, with both correctly migrating 89% of migration-related code changes respectively. Overall, our results suggest that using LLMs for library migration is promising, and we discuss the opportunities for further work in this area.

10 Data Availability

The data and replication package for this study is available at https://figshare.com/s/a3bd8ef8e13d7b2395f1.

References

[1] aio libs. 2024. aiohttp. https://pypi.org/project/aiohttp.

[2] Hussein Alrubaye and Mohamed Wiem Mkaouer. 2018. Automating the detection of third-party Java library migration at the function level.. In *CASCON*. 60–71.

[3] Hussein Alrubaye, Mohamed Wiem Mkaouer, Igor Khokhlov, Leon Reznik, Ali Ouni, and Jason Mcgoff. 2020. Learning to recommend third-party library migration opportunities at the API level. *Applied Soft Computing* 90 (2020), 106140.

[4] Hussein Alrubaye, Mohamed Wiem Mkaouer, and Ali Ouni. 2019. On the use of information retrieval to automate the detection of third-party java library migration at the method level. In *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*. IEEE, 347–357.

[5] Berk Atil, Alexa Chittams, Liseng Fu, Ferhan Ture, Lixinyu Xu, and Breck Baldwin. 2024. LLM Stability: A detailed analysis with some surprises. arXiv:2408.04667 [cs.CL] https://arxiv.org/abs/2408.04667

[6] Nélio Cacho, Eiji Adachi Barbosa, Juliana Araujo, Frederico Pranto, Alessandro Garcia, Thiago Cesar, Eliezio Soares, Arthur Cassio, Thomas Filipe, and Israel Garcia. 2014. How does exception handling behavior evolve? an exploratory study in java and c# applications. In *2014 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 31–40.

[7] Chunyang Chen. 2020. Similarapi: mining analogical apis for library migration. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings*. 37–40.

[8] Chunyang Chen, Zhenchang Xing, Yang Liu, and Kent Ong Long Xiong. 2019. Mining likely analogical apis across third-party libraries via large-scale unsupervised api semantics embedding. *IEEE Transactions on Software Engineering* 47, 3 (2019), 432–447.

[9] Chunyang Chen, Zhenchang Xing, Yang Liu, and Kent Ong Long Xiong. 2021. Mining Likely Analogical APIs Across Third-Party Libraries via Large-Scale Unsupervised API Semantics Embedding. *IEEE Transactions on Software Engineering* 47, 3 (2021), 432–447. https://doi.org/10.1109/TSE.2019.2896123

[10] Jeffrey A. Clark, Hugo van Kemenade, and Andrew Murray. 2024. PIL. https://pypi.org/project/PIL/.

[11] Jeffrey A. Clark, Hugo van Kemenade, and Andrew Murray. 2024. Pillow. https://python-pillow.org/.

[12] Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and psychological measurement* 20, 1 (1960), 37–46.

[13] The Git Development Community. 2024. git-diff. https://git-scm.com/docs/git-diff.

[14] Erik Derr, Sven Bugiel, Sascha Fahl, Yasemin Acar, and Michael Backes. 2017. Keep me updated: An empirical study of third-party library updatability on android. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2187–2200.

[15] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[16] Mikhail Evtikhiev, Egor Bogomolov, Yaroslav Sokolov, and Timofey Bryksin. 2023. Out of the bleu: how should we assess quality of the code generation models? *Journal of Systems and Software* 203 (2023), 111741.

[17] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. 2023. Large language models for software engineering: Survey and open problems. *arXiv preprint arXiv:2310.03533* (2023).

[18] Python Software Foundation. 2022. requests. https://pypi.org/project/requests.

[19] Python Software Foundation. 2024. argparse. https://docs.python.org/3/library/argparse.html.

[20] Python Software Foundation. 2024. ast. https://docs.python.org/3/library/ast.html.

[21] Python Software Foundation. 2024. ast. https://docs.python.org/3/library/ast.html#ast.unparse.

[22] Martin Fowler. 2018. *Refactoring: improving the design of existing code.* Addison-Wesley Professional.

[23] Rory Geoghegan. 2024. retrying. https://pypi.org/project/retrying.

[24] Zhenfei Huang, Junjie Chen, Jiajun Jiang, Yihua Liang, Hanmo You, and Fengjie Li. 2024. Mapping APIs in Dynamic-typed Programs by Leveraging Transfer Learning. *ACM Trans. Softw. Eng. Methodol.* (jan 2024). https://doi.org/10.1145/3641848 Just Accepted.

[25] Mohayeminul Islam, Ajay Kumar Jha, Ildar Akhmetov, and Sarah Nadi. 2024. Characterizing Python Library Migrations. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. Article 5, 23 pages. https://doi.org/10.1145/3643731

[26] Mohayeminul Islam, Ajay Kumar Jha, Sarah Nadi, and Ildar Akhmetov. 2023. PyMigBench: A Benchmark for Python Library Migration. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 511–515. https://doi.org/10.1109/MSR59073.2023.00075

[27] Raula Gaikovina Kula, Daniel M German, Ali Ouni, Takashi Ishio, and Katsuro Inoue. 2018. Do developers update their library dependencies? *Empirical Software Engineering* 23, 1 (2018), 384–417.

[28] J Richard Landis and Gary G Koch. 1977. The measurement of observer agreement for categorical data. *biometrics* (1977), 159–174.

[29] Meta. 2025. Meta Llama. https://llama.meta.com/.

[30] Vijayaraghavan Murali, Chandra Maddila, Imad Ahmad, Michael Bolin, Daniel Cheng, Negar Ghorbani, Renuka Fernandez, and Nachiappan Nagappan. 2023. CodeCompose: A large-scale industrial deployment of AI-assisted code authoring. *arXiv preprint arXiv:2305.12050* (2023).

[31] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an LLM to Help With Code Understanding. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 881–881.

[32] Nhan Nguyen and Sarah Nadi. 2022. An empirical evaluation of GitHub copilot's code suggestions. In *Proceedings of the 19th International Conference on Mining Software Repositories*. 1–5.

[33] Ansong Ni, Daniel Ramos, Aidan ZH Yang, Inês Lynce, Vasco Manquinho, Ruben Martins, and Claire Le Goues. 2021. Soar: a synthesis approach for data science api refactoring. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 112–124.

[34] OpenAI. 2024. OpenAI models. https://platform.openai.com/docs/models.

[35] OpenAI. 2025. OpenAI models: GPT-4o. https://platform.openai.com/docs/models/gpt-4o.

[36] OpenAI. 2025. OpenAI models: GPT-4o mini. https://platform.openai.com/docs/models/gpt-4o-mini.

[37] Ipek Ozkaya. 2023. Application of large language models to software engineering tasks: Opportunities, risks, and implications. *IEEE Software* 40, 3 (2023), 4–8.

[38] Pallets. 2024. click. https://pypi.org/project/click.

[39] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 311–318.

[40] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. 2023. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590* (2023).

[41] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* (2020).

[42] Johann Schmitz. 2024. tenacity. https://pypi.org/project/tenacity.

[43] Atsushi Shirafuji, Yusuke Oda, Jun Suzuki, Makoto Morishita, and Yutaka Watanobe. 2023. Refactoring Programs Using Large Language Models with Few-Shot Examples. *arXiv preprint arXiv:2311.11690* (2023).

[44] Davide Spadini, Maurício Aniche, and Alberto Bacchelli. 2018. PyDriller: Python framework for mining software repositories. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering - ESEC/FSE 2018*. ACM Press, New York, New York, USA, 908–911. https://doi.org/10.1145/3236024.3264598

[45] Cedric Teyton, Jean-Remy Falleri, and Xavier Blanc. 2012. Mining library migration graphs. In *2012 19th Working Conference on Reverse Engineering*. IEEE, 289–298.

[46] Cédric Teyton, Jean-Rémy Falleri, and Xavier Blanc. 2013. Automatic discovery of function mappings between similar libraries. In *2013 20th Working Conference on Reverse Engineering (WCRE)*. IEEE, 192–201.

[47] Chenglong Wang, Jiajun Jiang, Jun Li, Yingfei Xiong, Xiangyu Luo, Lu Zhang, and Zhenjiang Hu. 2016. Transforming Programs between APIs with Many-to-Many Mappings. In *30th European Conference on Object-Oriented Programming (ECOOP 2016) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 56)*, Shriram Krishnamurthi and Benjamin S. Lerner (Eds.). Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 25:1–25:26. https://doi.org/10.4230/LIPIcs.ECOOP.2016.25

[48] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering* (2024).

[49] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi DQ Bui, Junnan Li, and Steven CH Hoi. 2023. Codet5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922* (2023).

[50] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859* (2021).

[51] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the copilots: Fusing large language models with completion engines for automated program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 172–184.

[52] Kai Willadsen. 2024. Meld. https://meldmerge.org.

[53] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1482–1494.

[54] Shengzhe Xu, Ziqi Dong, and Na Meng. 2019. Meditor: inference and application of API migration edits. In *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*. IEEE, 335–346.

[55] Hongbin Ye, Tong Liu, Aijia Zhang, Wei Hua, and Weiqiang Jia. 2023. Cognitive mirage: A review of hallucinations in large language models. *arXiv preprint arXiv:2309.06794* (2023).

[56] Jiyang Zhang, Pengyu Nie, Junyi Jessy Li, and Milos Gligoric. 2023. Multilingual code co-evolution using large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 695–707.

[57] Zejun Zhang, Minxue Pan, Tian Zhang, Xinyu Zhou, and Xuandong Li. 2020. Deep-diving into documentation to develop improved java-to-swift api mapping. In *Proceedings of the 28th International Conference on Program Comprehension*. 106–116.

[58] Bingzhe Zhou, Xinying Wang, Shengbin Xu, Yuan Yao, Minxue Pan, Feng Xu, and Xiaoxing Ma. 2023. Hybrid API migration: A marriage of small API mapping models and large language models. In *Proceedings of the 14th Asia-Pacific Symposium on Internetware*. 12–21.

[59] Bingzhe Zhou, Xinying Wang, Shengbin Xu, Yuan Yao, Minxue Pan, Feng Xu, and Xiaoxing Ma. 2023. Hybrid API migration: A marriage of small API mapping models and large language models. In *Proceedings of the 14th Asia-Pacific Symposium on Internetware*. 12–21.